SPI Interface

By: Tarek Elashmawy

SPI Interface

```
interface SPI_if (clk);
  input clk;
  logic rst_n,tx_valid,rx_valid,MISO,MOSI,SS_n;
  logic [7:0] tx_data;
  logic [9:0] rx_data;

  logic [9:0] rx_data_ref;
  logic MISO_ref;
  logic rx_valid_ref;

  modport SVA_mp (
        input   clk, MOSI, rst_n, SS_n, tx_valid, tx_data,
        input   MISO, rx_valid, rx_data  // SVA monitors all signals
    );
endinterface : SPI_if
```

Run.do

```
vlib work
vlog -f src_files.list
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
add wave /top/SPI_if_inst/*
coverage save top.ucdb -onexit
run -all
```

```
SPI_if.sv
SPI_slave.sv
SVA.sv
golden_model.sv
sequence_item.sv
SPI_config.sv
sequencer.sv
monitor.sv
SPI_driver.sv
Agent.sv
scoreboard.sv
coverage_collector.sv
SPI_env.sv
main_seq.sv
reset_seq.sv
SPI_test.sv
top.sv
```

DUT

```
module SLAVE (
    input MOSI, clk, rst_n, SS_n, tx_valid,
    input [7:0] tx_data,
    output reg MISO, rx_valid,
    output reg [9:0] rx_data
);
    // State definitions - must match RTL exactly
    parameter IDLE      = 3'b000;
    parameter WRITE     = 3'b001;
    parameter CHK_CMD   = 3'b010;
    parameter READ_ADD  = 3'b011;
    parameter READ_DATA = 3'b100;

    reg [2:0] cs, ns;
    reg [3:0] counter;
    reg confirm_add;
    reg [9:0] rx_data_internal;

   (*fsm_encoding="one_hot"*)
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cs <= IDLE;
            counter <= 0;
            confirm_add <= 0;
            rx_data_internal <= 0;
            MISO <= 0;
            rx_valid <= 0;
        end
        else begin
           cs <= ns;

            // State behavior
            case (cs)
                IDLE: begin
                    counter <= 0;
                    rx_valid <= 0;
                    MISO <= 0;
                end

                WRITE: begin
                    counter <= 0 ;
                    if (counter < 10) begin
                        rx_data_internal <= {rx_data_internal[8:0], MOSI};
                        counter <= counter + 1;
                        rx_valid <= 0;
                    end
                    else if (counter == 10) begin
                        rx_valid <= 1;
                    end
                end

                READ_ADD: begin
                    if (counter < 10) begin
                        rx_data_internal <= {rx_data_internal[8:0], MOSI};
                        counter <= counter + 1;
                        rx_valid <= 0;
                    end
                    else if (counter == 10) begin
                        rx_valid <= 1;
                        confirm_add <= 1;
                    end
                end

                READ_DATA: begin
                    if (!tx_valid) begin
                        if (counter < 10) begin
                            rx_data_internal <= {rx_data_internal[8:0], MOSI};
                            counter <= counter + 1;
                            rx_valid <= 0;
                        end
                        else if (counter == 10) begin
                            rx_valid <= 1;
                            confirm_add <= 0;
                        end
                    end
                    else begin
                        counter <= 10;
                        if (counter >= 3 && counter <= 10) begin
                            MISO <= tx_data[counter-3];
                            counter <= counter - 1;
                        end
                    end
                end

                default: begin
                    // Safe default
                    counter <= 0;
                    rx_valid <= 0;
                    MISO <= 0;
                end
            endcase
        end
    end

    // Next state logic (combinational)
    always @(*) begin
        case (cs)
            IDLE: begin
                ns = (~SS_n) ? CHK_CMD : IDLE;
            end

            CHK_CMD: begin
                if (SS_n) begin
                    ns = IDLE;
                end
                else if (~SS_n && ~MOSI) begin
                    ns = WRITE;
                end
                else if (~SS_n && MOSI && ~confirm_add) begin
                    ns = READ_ADD;
                end
                else if (~SS_n && MOSI && confirm_add) begin
                    ns = READ_DATA;
                end
                else begin
                    ns = CHK_CMD;
                end
            end

            WRITE: begin
                ns = SS_n ? IDLE : WRITE;
            end

            READ_ADD: begin
                if (SS_n) begin
                    ns = IDLE;
                end
                else if (confirm_add) begin
                    ns = READ_DATA;
                end
                else begin
                    ns = READ_ADD;
                end
            end

            READ_DATA: begin
                ns = SS_n ? IDLE : READ_DATA;
            end

            default: ns = IDLE;
        endcase
    end

    // Output assignment
    assign rx_data = rx_data_internal;

```

SVA & Design Assertions

```
property A1;
    @(posedge clk) disable iff (!rst_n || SS_n) ((cs == WRITE || cs == READ_ADD || cs == READ_DATA) && (counter==10) && (!tx_valid && cs==READ_DATA)) |=> ##1 ##0 ($past(rx_valid) == 1);
endproperty

a1: assert property(A1);
cover property(A1);

property A2;
    @(posedge clk) disable iff (!rst_n) (cs == IDLE) |-> ##[1:$] (cs == CHK_CMD);
endproperty

a2: assert property(A2);
cover property(A2);

property A3;
    @(posedge clk) disable iff (!rst_n) (cs == CHK_CMD) |-> ##[1:$] ((cs == WRITE) || (cs == READ_ADD) || (cs == READ_DATA));
endproperty

a3: assert property(A3);
cover property(A3);

property A4;
    @(posedge clk) disable iff (!rst_n) (cs == WRITE) |-> ##[1:$] (cs == IDLE);
endproperty

a4: assert property(A4);
cover property(A4);

property A5;
    @(posedge clk) disable iff (!rst_n) (cs == READ_ADD) |-> ##[1:$] (cs == IDLE);
endproperty

a5: assert property(A5);
cover property(A5);

property A6;
    @(posedge clk) disable iff (!rst_n) (cs == READ_DATA) |-> ##[1:$] (cs == IDLE);
endproperty

a6: assert property(A6);
cover property(A6);
```

```
module SVA (SPI_if.SVA_mp vif);

always_comb begin
if (!vif.rst_n) begin
a_reset: assert final (vif.MISO == 1'b0 && vif.rx_data == 8'h0 && vif.rx_valid == 1'b0);
cover final (vif.MISO == 1'b0 && vif.rx_data == 8'h0 && vif.rx_valid == 1'b0);
end
end
endmodule
```

Golden Model

```
module golden_model (
    input MOSI, clk, rst_n, SS_n, tx_valid,
    input [7:0] tx_data,
    output reg MISO, rx_valid,
    output reg [9:0] rx_data
);
    // State definitions - must match RTL exactly
    parameter IDLE      = 3'b000;
    parameter WRITE     = 3'b001;
    parameter CHK_CMD   = 3'b010;
    parameter READ_ADD  = 3'b011;
    parameter READ_DATA = 3'b100;

    reg [2:0] cs, ns;
    reg [3:0] counter;
    reg confirm_add;
    reg [9:0] rx_data_internal;

   (*fsm_encoding="one_hot"*)
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cs <= IDLE;
            counter <= 0;
            confirm_add <= 0;
            rx_data_internal <= 0;
            MISO <= 0;
            rx_valid <= 0;
        end
        else begin
           cs <= ns;

            // State behavior
            case (cs)
                IDLE: begin
                    counter <= 0;
                    rx_valid <= 0;
                    MISO <= 0;
                end

                WRITE: begin
                    counter <= 0 ;
                    if (counter < 10) begin
                        rx_data_internal <= {rx_data_internal[8:0], MOSI};
                        counter <= counter + 1;
                        rx_valid <= 0;
                    end
                    else if (counter == 10) begin
                        rx_valid <= 1;
                    end
                end

                READ_ADD: begin
                    if (counter < 10) begin
                        rx_data_internal <= {rx_data_internal[8:0], MOSI};
                        counter <= counter + 1;
                        rx_valid <= 0;
                    end
                    else if (counter == 10) begin
                        rx_valid <= 1;
                        confirm_add <= 1;
                    end
                end

                READ_DATA: begin
                    if (!tx_valid) begin
                        if (counter < 10) begin
                            rx_data_internal <= {rx_data_internal[8:0], MOSI};
                            counter <= counter + 1;
                            rx_valid <= 0;
                        end
                        else if (counter == 10) begin
                            rx_valid <= 1;
                            confirm_add <= 0;
                        end
                    end
                    else begin
                        counter <= 10;
                        if (counter >= 3 && counter <= 10) begin
                            MISO <= tx_data[counter-3];
                            counter <= counter - 1;
                        end
                    end
                end

                default: begin
                    // Safe default
                    counter <= 0;
                    rx_valid <= 0;
                    MISO <= 0;
                end
            endcase
        end
    end

    // Next state logic (combinational)
    always @(*) begin
        case (cs)
            IDLE: begin
                ns = (~SS_n) ? CHK_CMD : IDLE;
            end

            CHK_CMD: begin
                if (SS_n) begin
                    ns = IDLE;
                end
                else if (~SS_n && ~MOSI) begin
                    ns = WRITE;
                end
                else if (~SS_n && MOSI && ~confirm_add) begin
                    ns = READ_ADD;
                end
                else if (~SS_n && MOSI && confirm_add) begin
                    ns = READ_DATA;
                end
                else begin
                    ns = CHK_CMD;
                end
            end

            WRITE: begin
                ns = SS_n ? IDLE : WRITE;
            end

            READ_ADD: begin
                if (SS_n) begin
                    ns = IDLE;
                end
                else if (confirm_add) begin
                    ns = READ_DATA;
                end
                else begin
                    ns = READ_ADD;
                end
            end

            READ_DATA: begin
                ns = SS_n ? IDLE : READ_DATA;
            end

            default: ns = IDLE;
        endcase
    end

    // Output assignment
    assign rx_data = rx_data_internal;

endmodule
```

Sequence Item

```
package sequence_item_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

class sequence_item extends uvm_sequence_item;
`uvm_object_utils(sequence_item)

rand logic MOSI;
rand logic SS_n;
rand logic tx_valid;
rand logic [7:0] tx_data;
rand bit rst_n;

logic rx_valid,rx_valid_ref;
logic [9:0] rx_data,rx_data_ref;
logic MISO,MISO_ref;

rand bit [10:0] mosi_array;  // 11-bit randomized array
int cycle_count = 0,read_data_cycle=0,cases_cycle=0;        // Cycle counter for SS_n timing
bit generate_new_array; // Flag to indicate when to generate a new array

function new(string name = "sequence_item");
super.new(name);
endfunction

//000 (Write Address)
//001 (Write Data)
//110 (Read Address)
//111 (Read Data)

constraint c1 {rst_n dist {1:=95, 0:=5};}
// Constraint 4: tx_valid high in case of read data
constraint c2 {tx_valid == (mosi_array[2:0]==3'b111);}

// Constraint for valid first 3 bits (applied during SS_n falling edge)
constraint valid_first_3_bits {
    // This constraint ensures that when SS_n falls, the first 3 bits are valid
    // The actual application timing would be handled in the driver/sequence
    if (!SS_n) mosi_array[2:0] inside {3'b000, 3'b001, 3'b110, 3'b111};
}

constraint array_regeneration {
    // Array gets new random values
    foreach (mosi_array[i]) {
        mosi_array[i] inside {[0:1]};
    }
}
    function void pre_randomize();
        // Determine if we need new array
        generate_new_array = (cycle_count % 11 == 0);
        // Control array randomization
        if (generate_new_array) begin
            mosi_array.rand_mode(1);  // Enable array randomization
            //$display("Generating NEW array at cycle %0d", cycle_count);
        end else begin
            mosi_array.rand_mode(0);  // Keep existing array values
            //$display("Reusing array at cycle %0d, index %0d", cycle_count, cycle_count % 11);
        end
    endfunction

// Post-randomize for points 2 and 3
function void post_randomize();
    // Point 3: Drive MOSI from the randomized array
    MOSI = mosi_array[cycle_count % 11];
    // Point 2: SS_n timing based on operation type
    if ((mosi_array[2:0]==3'b111)) begin
        // Read data: SS_n high for one cycle every 23 cycles
        SS_n = (read_data_cycle % 23 == 0) ? 1'b1 : 1'b0;
    end else begin
        // All other cases: SS_n high for one cycle every 13 cycles
        SS_n = (cases_cycle % 13 == 0) ? 1'b1 : 1'b0;
    end
    // Increment cycle counter for next transaction
    cycle_count++;
    if ((mosi_array[2:0]==3'b111)) read_data_cycle++;
    else cases_cycle++;
endfunction

function string convert2string();
return $sformatf("%s rst_n=%b tx_valid=%d MOSI=%b tx_data=%b SS_n=%d rx_data=%b MISO=%d",super.convert2string(),rst_n,tx_valid,MOSI,tx_data,SS_n,rx_data,MISO);
endfunction
function string convert2string_stimulus();
return $sformatf("rst_n=%b tx_valid=%d MOSI=%b tx_data=%b SS_n=%d rx_data=%b MISO=%d",rst_n,tx_valid,MOSI,tx_data,SS_n,rx_data,MISO);
endfunction
endclass
```

SPI Config

```
package SPI_config_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

class SPI_config extends uvm_object;
  virtual SPI_if vif;
  `uvm_object_utils(SPI_config)

  function new(string name = "SPI_config");
    super.new(name);
  endfunction
endclass



endpackage
```

Sequencer

```
package sequencer_pkg;
import uvm_pkg::*;
import sequence_item_pkg::*;
`include "uvm_macros.svh"

class sequencer extends uvm_sequencer #(sequence_item);
`uvm_component_utils(sequencer)

function new(string name = "sequencer",uvm_component parent = null);
super.new(name,parent);
endfunction

endclass

endpackage
```

Monitor

```
package monitor_pkg;
import uvm_pkg::*;
//import shared_pkg::*;
import sequence_item_pkg::*;
`include "uvm_macros.svh"

class monitor extends uvm_monitor;
`uvm_component_utils(monitor)

virtual SPI_if vif;
sequence_item seq_item;
uvm_analysis_port #(sequence_item) ap;

function new(string name="monitor", uvm_component parent = null);
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
ap = new("ap",this);
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
    seq_item = sequence_item::type_id::create("seq_item");
    @(negedge vif.clk);
    seq_item.rst_n = vif.rst_n;
    seq_item.tx_valid = vif.tx_valid;
    seq_item.tx_data = vif.tx_data;
    seq_item.MOSI = vif.MOSI;
    seq_item.SS_n = vif.SS_n;
    seq_item.MISO = vif.MISO;
    seq_item.rx_data = vif.rx_data;
    seq_item.rx_valid = vif.rx_valid;

    seq_item.rx_data_ref = vif.rx_data_ref;
    seq_item.MISO_ref = vif.MISO_ref;
    seq_item.rx_valid_ref = vif.rx_valid_ref;
    ap.write(seq_item);
    `uvm_info("run_phase",seq_item.convert2string(),UVM_HIGH);
end
endtask
endclass

endpackage
```

Driver

```
package SPI_driver_pkg;
import SPI_config_pkg::*;
import sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class SPI_driver extends uvm_driver #(sequence_item);
  `uvm_component_utils(SPI_driver)  // Added parentheses around class name

  virtual SPI_if vif;
  SPI_config cfg;
  sequence_item stim_seq_item;

  function new (string name="SPI_driver",uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(SPI_config)::get(this,"","CFG",cfg)) begin  // Fixed type parameter
        `uvm_fatal("DRIVER","unable to get config object")
    end
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    vif = cfg.vif;
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);

    forever begin
        stim_seq_item = sequence_item::type_id::create("stim_seq_item");
        seq_item_port.get_next_item(stim_seq_item);
        vif.MOSI= stim_seq_item.MOSI;
        vif.tx_valid = stim_seq_item.tx_valid;
        vif.tx_data = stim_seq_item.tx_data;
        vif.rst_n = stim_seq_item.rst_n;
        vif.SS_n = stim_seq_item.SS_n;
        @(negedge vif.clk);
        seq_item_port.item_done();
        `uvm_info("run_phase",stim_seq_item.convert2string(),UVM_HIGH);
    end
  endtask

endclass

endpackage
```

Agent

```
package agent_pkg;
import uvm_pkg::*;
import sequence_item_pkg::*;
import SPI_driver_pkg::*;
import monitor_pkg::*;
import SPI_config_pkg::*;
import sequencer_pkg::*;
`include "uvm_macros.svh"

class agent extends uvm_agent;
`uvm_component_utils(agent)

sequencer sqr;
SPI_driver driver;
monitor mon;
SPI_config cfg;
uvm_analysis_port #(sequence_item) ap;

function new (string name = "agent",uvm_component parent = null);
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
if (!uvm_config_db #(SPI_config)::get(this,"","CFG",cfg))
    `uvm_fatal("build_phase","unable to get config object")

    sqr = sequencer::type_id::create("sqr",this);
    driver = SPI_driver::type_id::create("driver",this);
    mon = monitor::type_id::create("mon",this);
    ap = new("ap",this);

endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
driver.vif = cfg.vif;
mon.vif = cfg.vif;
driver.seq_item_port.connect(sqr.seq_item_export);
mon.ap.connect(ap);

endfunction


endclass
endpackage
```

Scoreboard

```
package scoreboard_pkg;
import uvm_pkg::*;
import sequence_item_pkg::*;
`include "uvm_macros.svh"

class scoreboard extends uvm_scoreboard;
`uvm_component_utils(scoreboard)

uvm_analysis_export #(sequence_item) sb_export;
uvm_tlm_analysis_fifo #(sequence_item) sb_fifo;
sequence_item seq_item_sb;

int error_count=0;
int correct_count=0;

function new (string name="scoreboard", uvm_component parent = null);
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
sb_export = new("sb_export",this);
sb_fifo = new("sb_fifo",this);

endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
sb_export.connect(sb_fifo.analysis_export);

endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
    sb_fifo.get(seq_item_sb);

    if (seq_item_sb.rx_data != seq_item_sb.rx_data_ref ||
    seq_item_sb.rx_valid != seq_item_sb.rx_valid_ref || seq_item_sb.MISO != seq_item_sb.MISO_ref) begin

        `uvm_error("run_phase",$sformatf("comparison failed: %s while ref=%d",seq_item_sb.convert2string(),seq_item_sb.rx_data_ref));
        error_count++;
    end else correct_count++;
end
endtask


function void report_phase(uvm_phase phase);
super.report_phase(phase);
`uvm_info("report_phase",$sformatf("Total correct transactions=%0d",correct_count),UVM_LOW);
`uvm_info("report_phase",$sformatf("Total error transactions=%0d",error_count),UVM_LOW);

endfunction
endclass
endpackage
```

Coverage Collector

```
package cover_collect_pkg;
import uvm_pkg::*;
import sequence_item_pkg::*;
`include "uvm_macros.svh"

class cover_collect extends uvm_component;
`uvm_component_utils(cover_collect)
uvm_analysis_export #(sequence_item) cov_export;
uvm_tlm_analysis_fifo #(sequence_item) cov_fifo;
sequence_item cov_seq;

covergroup cov_grp;
rx_data: coverpoint cov_seq.rx_data[9:8]{
    bins values[] = {[0:3]};
    // Cover all possible transitions (more comprehensive)
    bins trans[] = (0,1,2,3 => 0,1,2,3);
    ignore_bins i_tran = (0 => 2), (0 => 3), (2 => 3), (3 => 1);

}

ss_n: coverpoint cov_seq.SS_n {
    bins normal_transaction1 = (1 => 0 [*13] => 1);
    bins normal_transaction2 = (1 => 0 [*23] => 1);
}

//000 (Write Address)
//001 (Write Data)
//110 (Read Address)
//111 (Read Data)

mosi_command_transitions: coverpoint cov_seq.MOSI {
        bins write_addr = (0 => 0 => 0);
        bins write_data = (1 => 0 => 0);
        bins read_addr  = (0 => 1 => 1);
        bins read_data  = (1 => 1 => 1);
    }

cp1: coverpoint cov_seq.MOSI{
    bins low = {0};
    bins high = {1};
    option.weight=0;
}

cp2: coverpoint cov_seq.SS_n{
    bins low = {0};
    bins high = {1};
    option.weight=0;
}

cross_: cross cp1, cp2  {
    bins correct_combs = binsof(cp1) && binsof(cp2.low);
    option.cross_auto_bin_max=0;
}
endgroup

function new(string name = "cover_collect",uvm_component parent = null);
super.new(name,parent);
cov_grp = new();
endfunction

function void build_phase (uvm_phase phase);
super.build_phase(phase);
cov_export = new("cov_export",this);
cov_fifo = new("cov_fifo",this);
endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase (uvm_phase phase);
super.run_phase(phase);
forever begin

    cov_fifo.get(cov_seq);
    cov_grp.sample();
end
endtask
endclass
endpackage
```

Spi Env

```
package SPI_env_pkg;
import uvm_pkg::*;
import SPI_config_pkg::*;
import agent_pkg::*;
import scoreboard_pkg::*;
import cover_collect_pkg::*;
`include "uvm_macros.svh"

class SPI_env extends uvm_env;
  // Example 1
  // Do the essentials (factory register & Constructor)
  `uvm_component_utils(SPI_env)

  agent agt;
  scoreboard sb;
  cover_collect cc;

  function new(string name = "SPI_env", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = agent::type_id::create("agt",this);
    sb = scoreboard::type_id::create("sb",this);
    cc = cover_collect::type_id::create("cc",this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    agt.ap.connect(sb.sb_export);
    agt.ap.connect(cc.cov_export);
  endfunction

endclass
endpackage
```

Main Sequence

```
package main_seq_pkg;
import uvm_pkg::*;
//import shared_pkg::*;
import sequence_item_pkg::*;
`include "uvm_macros.svh"

class main_seq extends uvm_sequence #(sequence_item);
`uvm_object_utils(main_seq)

sequence_item sq_item;

function new(string name = "main_seq");
super.new();
endfunction

task body;

sq_item = sequence_item::type_id::create("sq_item");

repeat(10000) begin
start_item(sq_item);
assert(sq_item.randomize());
finish_item(sq_item);
end

endtask

endclass

endpackage
```

Reset Sequence

```
package reset_seq_pkg;
import uvm_pkg::*;
//import shared_pkg::*;
import sequence_item_pkg::*;
`include "uvm_macros.svh"

class reset_seq extends uvm_sequence #(sequence_item);
`uvm_object_utils(reset_seq)
sequence_item sq_item;

function new(string name = "reset_seq");
super.new();
endfunction

task body;

sq_item = sequence_item::type_id::create("sq_item");
start_item(sq_item);
sq_item.rst_n = 0;
sq_item.MOSI = 0;
sq_item.SS_n= 0;
sq_item.tx_valid = 0;
sq_item.tx_data = 0;
finish_item(sq_item);

endtask

endclass

endpackage
```

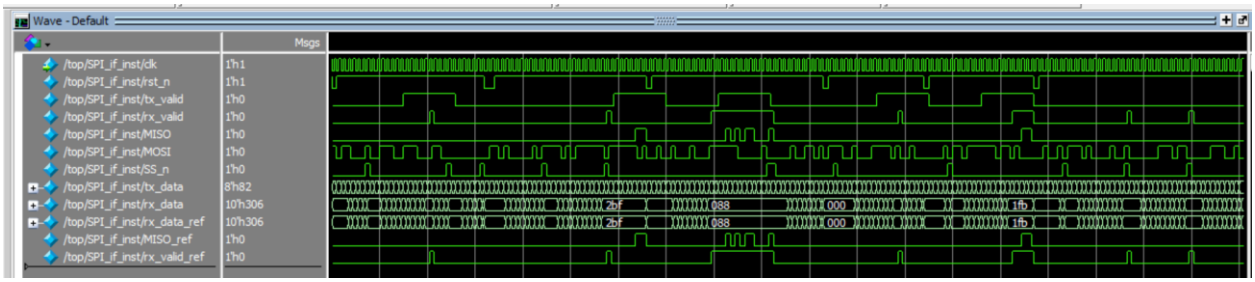
SPI Test

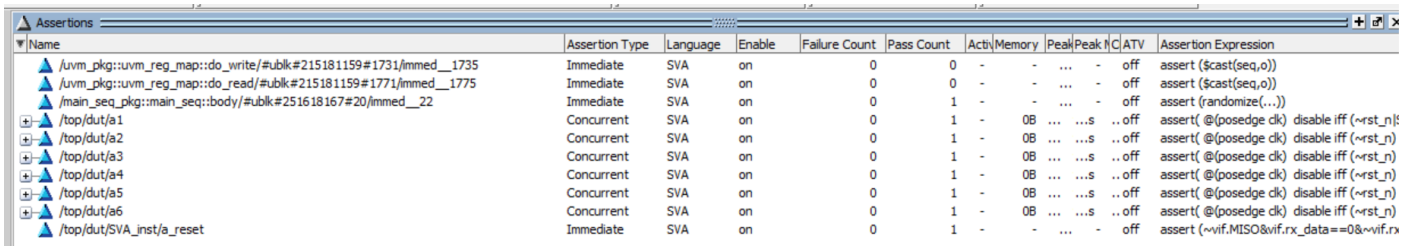
```
package SPI_test_pkg;
import SPI_env_pkg::*;
import SPI_config_pkg::*;
import reset_seq_pkg::*;
import main_seq_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class SPI_test extends uvm_test;
    `uvm_component_utils(SPI_test)

    SPI_config cfg;
    SPI_env my_env;
    main_seq main_sequence;
    reset_seq reset_sequence;

  // Example 1
  // Do the essentials (factory register & Constructor)
  // Build the enviornment in the build phase
  function new(string name = "SPI_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    my_env = SPI_env::type_id::create("my_env", this);
    main_sequence = main_seq::type_id::create("main_sequence");
    reset_sequence = reset_seq::type_id::create("reset_sequence");

    // Build the config object in the build phase
    cfg = SPI_config::type_id::create("cfg");

    // get the virtual interface and assign it to the virtual interface of the config object
    if(!uvm_config_db#(virtual SPI_if)::get(this, "", "vif", cfg.vif)) begin
      `uvm_error("NOVIF", "Virtual interface not found")
    end

    // set the config obj in the config db
    uvm_config_db#(SPI_config)::set(this, "*", "CFG", cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);

    `uvm_info("TEST", "Starting SPI test", UVM_LOW)
    reset_sequence.start(my_env.agt.sqr);
    `uvm_info("TEST", "Reset sequence completed", UVM_LOW)

    `uvm_info("TEST", "Starting main sequence", UVM_LOW)
    main_sequence.start(my_env.agt.sqr);
    `uvm_info("TEST", "Main sequence completed", UVM_LOW)
    phase.drop_objection(this);
  endtask

endclass: SPI_test
endpackage
```

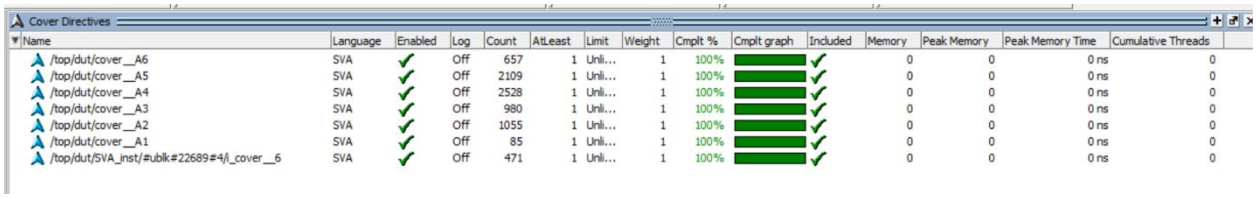
Top

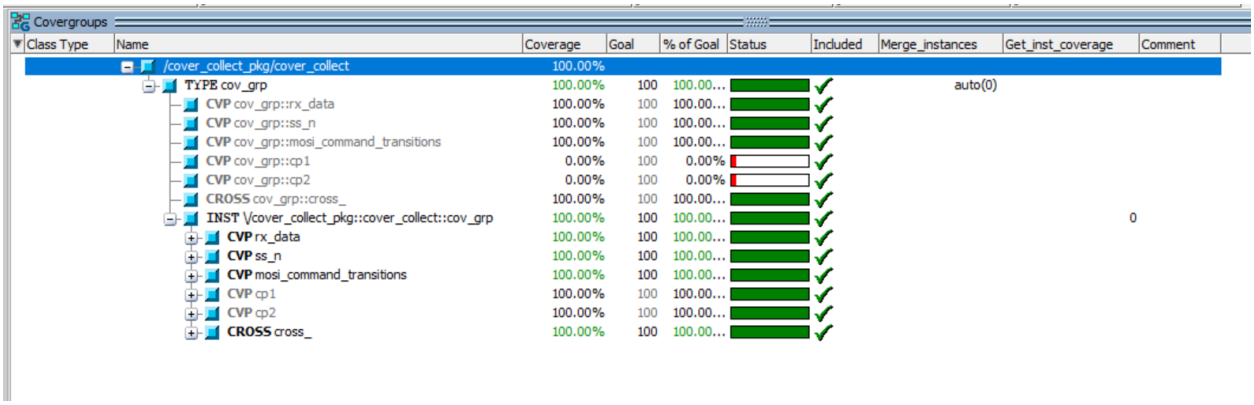
```
import uvm_pkg::*;
`include "uvm_macros.svh"
import SPI_test_pkg::*;

module top();
  // Clock generation
  bit clk;
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  // Instantiate the interface and DUT
  SPI_if SPI_if_inst(clk);
  SLAVE dut(
    .clk(clk),
    .MOSI(SPI_if_inst.MOSI),
    .rst_n(SPI_if_inst.rst_n),
    .SS_n(SPI_if_inst.SS_n),
    .tx_valid(SPI_if_inst.tx_valid),
    .tx_data(SPI_if_inst.tx_data),
    .MISO(SPI_if_inst.MISO),
    .rx_valid(SPI_if_inst.rx_valid),
    .rx_data(SPI_if_inst.rx_data)
  );

  golden_model ref_model(
    .clk(clk),
    .MOSI(SPI_if_inst.MOSI),
    .rst_n(SPI_if_inst.rst_n),
    .SS_n(SPI_if_inst.SS_n),
    .tx_valid(SPI_if_inst.tx_valid),
    .tx_data(SPI_if_inst.tx_data),
    .MISO(SPI_if_inst.MISO_ref),
    .rx_valid(SPI_if_inst.rx_valid_ref),
    .rx_data(SPI_if_inst.rx_data_ref)
  );

  bind SLAVE SVA SVA_inst(.vif(SPI_if_inst.SVA_mp));

  initial begin
    // Set the virtual interface for the uvm test
    uvm_config_db#(virtual SPI_if)::set(null, "*", "vif", SPI_if_inst);
    run_test("SPI_test");
  end

endmodule
```

Waveform & Reports










```
# *  To turn off, set 'recording_detail' to off:                          *
# * uvm_config_db#(int)            ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# ************************************************************************
# UVM_INFO SPI_test.sv(48) @ 10: uvm_test_top [TEST] Reset sequence completed
# UVM_INFO SPI_test.sv(50) @ 10: uvm_test_top [TEST] Starting main sequence
# UVM_INFO SPI_test.sv(52) @ 100010: uvm_test_top [TEST] Main sequence completed
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 100010: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO scoreboard.sv(50) @ 100010: uvm_test_top.my_env.sb [report_phase] Total correct transactions=10001
# UVM_INFO scoreboard.sv(51) @ 100010: uvm_test_top.my_env.sb [report_phase] Total error transactions=0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   10
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST]     4
# [TEST_DONE]     1
# [report_phase]     2
# ** Note: $finish    : D:/questasim_2021/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 100010 ns  Iteration: 61  Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at D:/questasim_2021/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```